pygeoinf

Technical Manual

Theory, Algorithms, and API Reference

Adrian

Department of Earth Sciences

February 27, 2026

Preface

Who this manual is for

This manual is written for geophysicists who have a forward operator and a dataset and want to perform rigorous linear inversion or inference. The word *rigorous* carries weight here: pygeoinf does not cut corners. It insists on correct Hilbert space structure, proper adjoints, and treatment of uncertainty that is mathematically honest. If you are used to inverting $(\mathbf{G}^\top\mathbf{G})^{-1}\mathbf{G}^\top$ and calling it done, this manual will explain why that is sometimes wrong, and what to do instead.

No prior knowledge of functional analysis is assumed, but you will need to engage with the mathematics. The package deliberately surfaces conceptual machinery — Riesz maps, dual pairings, Hilbert adjoints — that many inversion codes hide. The reward is a library whose results are correct by construction for *any* choice of model space, inner product, and forward operator, not just the Euclidean special case.

How to read this manual

Each chapter follows a consistent four-part structure:

1. *Why this matters* — a motivating argument, usually with a geophysical example, that explains why the concept is worth the effort. Highlighted in yellow callout boxes.
2. *Definitions and theorems* — the full, rigorous mathematical treatment.
3. *Implementation in pygeoinf* — how each mathematical concept maps to the API. Highlighted in blue callout boxes labelled "pygeoinf API".
4. *Worked example* — a complete mini-example, in both mathematics and code. Highlighted in green callout boxes.

If you are primarily a practitioner, you can read the yellow and blue boxes and skip the proofs on a first pass without losing the ability to use the package. If you are mathematically inclined, read everything; proofs that would interrupt the flow are deferred to Appendix C.

Red callout boxes labelled "Common Pitfall" flag mistakes that are easy to make and important to avoid.

Acknowledgements

This manual documents work carried out at the Department of Earth Sciences. The theoretical framework draws heavily on Al-Attar and Crawford [1], Stuart [5], and the trilogy Backus [2, 3, 4].

Contents

Preface **iii**

Notation and Conventions **ix**

I Foundations **1**

1 Introduction and Motivation **3**
1.1 What is pygeoinf? . . . 3
1.2 The geophysical inversion problem . . . 3
1.3 Why abstract spaces matter . . . 4
1.4 Package architecture overview . . . 5
1.5 How to read this manual . . . 6
1.6 Notation and conventions summary . . . 6

2 Hilbert Spaces **9**
2.1 Why Hilbert spaces? . . . 9
2.2 Definition and axioms . . . 9
2.3 Inner products, norms, and distances . . . 10
2.4 The dual space and dual pairing . . . 10
2.5 The Riesz representation theorem . . . 11
2.6 Bases and representations . . . 12
2.6.1 Riesz bases and biorthogonal duals . . . 12
2.6.2 The analysis operator . . . 13
2.6.3 The synthesis operator . . . 13
2.6.4 The dual basis . . . 13
2.7 The Gram matrix . . . 13
2.8 Finite-dimensional approximation . . . 14
2.9 Concrete spaces in pygeoinf . . . 15
2.9.1 `EuclideanSpace` . . . 15
2.9.2 `MassWeightedHilbertSpace` . . . 15
2.9.3 `HilbertSpaceDirectSum` . . . 15
2.9.4 Implementing your own `HilbertSpace` . . . 16
2.10 Summary and commutative diagram . . . 17

3 Linear Forms **19**
3.1 Motivation: measurements as linear functionals . . . 19

3.2 Definition . . . 19
3.3 Representation in coordinates . . . 19
3.4 The Riesz representative (gradient) . . . 19
3.5 Linear forms as rank-1 operators . . . 19

4 Linear Operators **21**
4.1 Motivation: the forward operator . . . 21
4.2 Definition and properties . . . 21
4.3 The Banach dual (transpose) G' . . . 21
4.4 The Hilbert adjoint G^* . . . 21
4.5 Matrix representations . . . 21
4.5.1 The standard (component) matrix . . . 21
4.5.2 The Galerkin matrix . . . 21
4.5.3 The discrete adjoint formula . . . 21
4.6 Operators as stacks of linear forms . . . 21
4.7 Kernel and range . . . 22
4.8 Constructing operators in pygeoinf . . . 22
4.9 The full commutative diagram . . . 22

5 Nonlinear Operators and Forms **23**
5.1 NonLinearOperator . . . 23
5.2 NonLinearForm . . . 23
5.3 Gradients and Hessians . . . 23

II Geometry and Structure **25**

6 Subsets and Convex Sets **27**
6.1 Why convex sets? . . . 27
6.2 The Subset interface . . . 27
6.3 Balls and ellipsoids . . . 27
6.4 Half-spaces and polyhedral sets . . . 27
6.5 Degenerate sets: EmptySet and FullSpace . . . 27
6.6 Set operations . . . 27

7 Subspaces **29**
7.1 Motivation: equality constraints . . . 29
7.2 Orthogonal projectors . . . 29
7.3 Linear subspaces . . . 29
7.4 Affine subspaces . . . 29
7.4.1 Construction from tangent bases . . . 29
7.4.2 Construction from linear equations . . . 29
7.5 Tangent spaces and dimension . . . 29

8 Convex Analysis and Support Functions **31**
8.1 Why convex analysis? . . . 31
8.2 Support functions: intuition . . . 32
8.3 Formal definition and properties . . . 32
8.4 Subgradients and support points . . . 32

8.5 Concrete support functions . . . 32
8.5.1 BallSupportFunction . . . 32
8.5.2 EllipsoidSupportFunction . . . 32
8.5.3 PolyhedralSupportFunction . . . 32
8.5.4 SobolevBallSupportFunction . . . 32
8.6 Combinators: building complex sets from simple ones . . . 32
8.6.1 Infimal convolution (Minkowski sum) . . . 32
8.6.2 Support function sum (intersection) . . . 32
8.6.3 Scaled sums . . . 32
8.7 The dual master equation . . . 32

9 Gaussian Measures **33**
9.1 Why Gaussian measures? . . . 33
9.2 Gaussian measures on Hilbert spaces . . . 33
9.3 Sampling . . . 33
9.4 Push-forward and marginals . . . 33
9.5 Conditioning (Bayesian update) . . . 33

III Problems and Algorithms **35**

10 Forward Problems **37**
10.1 The forward modelling framework . . . 37
10.2 ForwardProblem and LinearForwardProblem . . . 37

11 Deterministic Inversion **39**
11.1 Least-squares inversion . . . 39
11.2 Minimum-norm inversion . . . 39
11.3 Constrained inversions . . . 39
11.4 Connection to the dual master equation . . . 39

12 Bayesian Inversion **41**
12.1 The Bayesian framework for inverse problems . . . 41
12.2 Linear Bayesian inversion . . . 41
12.3 Constrained Bayesian inversion . . . 41
12.4 Posterior uncertainty . . . 41

13 Backus–Gilbert and DLI Inference **43**
13.1 The inference problem . . . 43
13.2 Backus–Gilbert estimators . . . 43
13.3 The admissible property set $\mathcal{U}$ . . . 43
13.4 Computing $\mathcal{U}$ via the dual master equation . . . 43
13.5 Polytope outer approximation . . . 43

14 Convex Optimisation **45**
14.1 Overview of optimisation in pygeoinf . . . 45
14.2 Linear solvers . . . 45
14.3 Convex optimisation algorithms . . . 45
14.4 Preconditioners . . . 45

IV Computational Methods and Visualisation **47**

15 Computational Utilities **49**
15.1 Randomised matrix decompositions . . . 49
15.2 Direct sum operators . . . 49
15.3 Parallel computation . . . 49
15.4 Auxiliary functions . . . 49

16 Visualisation **51**
16.1 The SubspaceSlicePlotter . . . 51
16.2 1D slices . . . 51
16.3 2D slices . . . 51
16.4 3D slices . . . 51
16.5 Worked example: visualising the admissible property set . . . 51

Appendices **55**

A Convex Analysis Primer **55**
A.1 Convex sets and functions . . . 55
A.2 Separation theorems . . . 55
A.3 Conjugate (Fenchel–Legendre) duality . . . 55
A.4 Subdifferentials . . . 55
A.5 Infimal convolution . . . 55

B Functional Analysis Prerequisites **57**
B.1 Banach spaces . . . 57
B.2 Weak and weak-* topologies . . . 57
B.3 Compact operators . . . 57
B.4 Spectral theory for self-adjoint operators . . . 57

C Deferred Proofs **59**

D Complete API Quick Reference **61**
D.1 hilbert_space module . . . 61
D.2 linear_forms module . . . 61
D.3 linear_operators module . . . 61
D.4 subsets module . . . 61
D.5 subspaces module . . . 61
D.6 convex_analysis module . . . 61
D.7 gaussian_measure module . . . 61
D.8 forward_problem module . . . 61
D.9 inversion module . . . 61
D.10 linear_bayesian module . . . 61
D.11 linear_optimisation module . . . 61
D.12 backus_gilbert module . . . 61
D.13 visualisation module . . . 61

Notation and Conventions

General conventions

Throughout this manual we use the following typographic conventions.

- **Boldface roman** ($\mathbf{m}$, $\mathbf{G}$, $\boldsymbol{\lambda}$) denotes finite-dimensional vectors and matrices, i.e. elements of $\mathbb{R}^n$ or linear maps $\mathbb{R}^n \to \mathbb{R}^m$.
- **Calligraphic** ($\mathcal{M}$, $\mathcal{D}$, $\mathcal{P}$) denotes abstract (possibly infinite-dimensional) Hilbert or Banach spaces.
- **Script/Roman capitals** (G, T, Π, π, $\mathcal{R}$) denote abstract linear operators between such spaces.
- Indices: j, k, ℓ typically index a basis of the model space; i typically indexes a datum.
- We work exclusively over $\mathbb{R}$; all spaces are real.

Spaces

Symbol	Meaning	Code
$\mathcal{M}$	Model space (Hilbert)	`model_space`
$\mathcal{D}$	Data space (usually $\mathbb{R}^{N_d}$)	`data_space`
$\mathcal{P}$	Property space (usually $\mathbb{R}^{N_p}$)	`property_space`
$\mathcal{M}'$	Continuous dual of $\mathcal{M}$	implicit via Riesz
$\mathcal{D}'$	Continuous dual of $\mathcal{D}$	implicit via Riesz
H	Generic Hilbert space	`HilbertSpace`
ℓ^2	Square-summable sequences	infinite-dim. limit
$\mathbb{R}^N$	Finite coefficient space	`EuclideanSpace(N)`
$\mathcal{M}_N$	N-dimensional subspace of $\mathcal{M}$	`dim = N`
$H_1 \oplus H_2$	Hilbert direct sum	`HilbertSpaceDirectSum`

Operators and maps

Symbol	Meaning	Code
$G : \mathcal{M} \to \mathcal{D}$	Forward operator	`forward_operator`
$G' : \mathcal{D}' \to \mathcal{M}'$	Banach dual (transpose)	`.dual`

$G^* : \mathcal{D} \to \mathcal{M}$	Hilbert adjoint	`.adjoint`
$\mathcal{T} : \mathcal{M} \to \mathcal{P}$	Property operator	`property_operator`
$\mathcal{R}_X : X' \to X$	Riesz isomorphism	`from_dual`
$\mathcal{R}_X^{-1} : X \to X'$	Inverse Riesz	`to_dual`
$\Pi : \mathcal{M} \to \ell^2$	Analysis (expansion coefficients)	`to_components`
$\pi : \ell^2 \to \mathcal{M}$	Synthesis (reconstruct from coeffs)	`from_components`
$\Pi_N : \mathcal{M} \to \mathbb{R}^N$	Truncated analysis	`to_components` (finite)
$\pi_N : \mathbb{R}^N \to \mathcal{M}_N$	Truncated synthesis	`from_components` (finite)
$P_N : \mathcal{M} \to \mathcal{M}_N$	Orthogonal projection onto $\mathcal{M}_N$	implicit
$\phi_j \in \mathcal{M}$	Basis vector	`basis_vector(j)`
$\psi_j \in \mathcal{M}$	Biorthogonal dual basis vector	derived from Gram

Inner products and pairings

Symbol	Meaning	Code
$(m_1, m_2)_{\mathcal{M}}$	Inner product on $\mathcal{M}$	`inner_product(x1,x2)`
$(d_1, d_2)_{\mathcal{D}}$	Inner product on $\mathcal{D}$	`inner_product(d1,d2)`
$\langle \ell, m \rangle$	Dual pairing: $\ell \in \mathcal{M}'$, $m \in \mathcal{M}$	`l(m)`
$\lVert m \rVert_{\mathcal{M}}$	Hilbert norm $\sqrt{(m,m)_{\mathcal{M}}}$	`norm(m)`

We use *parentheses* $(\cdot, \cdot)$ for inner products and *angle brackets* $\langle \cdot, \cdot \rangle$ for dual pairings. On a Hilbert space the two are related by $\langle \ell, m \rangle = (m, \mathcal{R}_X \ell)_X$.

Matrices (finite-dimensional objects)

Symbol	Meaning	Notes
$\mathbf{M}_{\phi,N}$	Gram/mass matrix $[\,(\phi_k, \phi_j)_{\mathcal{M}}\,]_{j,k}$	SPD, $N \times N$
$\mathbf{G}$	Standard discrete matrix of G: $[\,(G\phi_j, \beta_i)_{\mathcal{D}}\,]$	$N_d \times N$
$\tilde{\mathbf{G}}$	Galerkin matrix (with inner products)	`.matrix(galerkin=True)`
$\mathbf{m}, \mathbf{d}$	Coefficient vectors in $\mathbb{R}^N$, $\mathbb{R}^{N_d}$	`to_components(m)`

Measures and probability

Symbol	Meaning	Code
μ	A probability measure on $\mathcal{M}$	
$\mathcal{N}(\mu_0, C)$	Gaussian measure, mean μ_0, covariance C	`GaussianMeasure`
μ^0	Prior measure	`prior`
$\mu^{\tilde{d}}$	Posterior measure given data $\tilde{d}$	`.invert(data)`
$C : \mathcal{M} \to \mathcal{M}$	Covariance operator	`.covariance_operator`
$\mathbf{C}_{\mathcal{D}} : \mathcal{D} \to \mathcal{D}$	Data noise covariance	`data_error_measure`
$\varepsilon \in \mathcal{D}$	Data noise, $\varepsilon \sim \mathcal{N}(0, \mathbf{C}_{\mathcal{D}})$	

Convex analysis

Symbol	Meaning	Code
$C \subset \mathcal{M}$	A convex subset	`Subset` subclass
$h_C(q) = \sigma_C(q)$	Support function of C	`SupportFunction`
$\partial h_C(q)$	Subdifferential of h_C at q	`.subgradient(q)`
$\mathcal{U} \subset \mathcal{P}$	Admissible property set	computed by DLI

Part I

Foundations

Chapter 1

Introduction and Motivation

1.1 What is pygeoinf?

pygeoinf is a Python library for rigorous linear geophysical inversion and Bayesian inference. It is built around a single design philosophy: *the mathematical structure of the problem should be encoded explicitly in the code*, not reduced to bare matrices and vectors.

Most inversion codes operate on the discrete level from the start: the user assembles a matrix $\mathbf{G} \in \mathbb{R}^{M \times N}$, a data vector $\mathbf{d} \in \mathbb{R}^M$, and calls some solver. This approach works, but it forces the user to manually track which inner products are implicit, which transposes are adjoints, and how uncertainty propagates through the discretization. When anything non-standard happens — a non-Euclidean norm, a non-uniform grid, a Sobolev prior — the bookkeeping becomes error-prone.

Instead, pygeoinf exposes the following conceptual objects as first-class Python types:

- **Hilbert spaces** (`HilbertSpace`) — carrying their inner product, Riesz maps, and basis structure.
- **Linear operators** (`LinearOperator`) — aware of their domain and codomain, automatically computing the correct Hilbert adjoint from the spaces' Riesz maps.
- **Linear forms** (`LinearForm`) — elements of the dual space, representing individual measurements.
- **Gaussian measures** (`GaussianMeasure`) — priors and posteriors as probability measures on Hilbert spaces.
- **Convex sets and support functions** — geometry of the admissible model set.

Once these objects are defined, a full suite of inversion algorithms — Bayesian, least-squares, minimum-norm, Backus-Gilbert, and deterministic linear inference — work automatically and correctly for any choice of spaces and operators.

1.2 The geophysical inversion problem

The starting point is a physical system described by a *model* m — perhaps the seismic wave-speed field $v_P(x)$, the electrical conductivity $\sigma(x)$, or the subsurface density $\rho(x)$. Some aspects of this

system are measurable: we collect *data* $\tilde{d} = (\tilde{d}_1, \ldots, \tilde{d}_M)$ via instruments (travel-time picks, gravity readings, magnetic measurements, ...). A *forward model* G maps a model to its predicted data:

$$d = Gm + \varepsilon, \tag{1.1}$$

where ε is measurement noise. The *inverse problem* is to determine what equation (1.1) implies about m given the observed data $\tilde{d}$.

All realistic geophysical inverse problems share three features:

1. **Underdetermination.** The data $\tilde{d}$ is a finite-dimensional vector, whereas m is a function (infinite-dimensional in principle). Infinitely many models are consistent with the data exactly; the problem has a non-trivial null space.
2. **Ill-conditioning.** Even the components of m that *are* constrained by the data may be constrained very weakly, amplifying noise in the solution.
3. **Incomplete prior knowledge.** We have partial information about m — physical bounds, smoothness expectations, perhaps a statistical prior — that should inform the inversion.

Any honest inversion framework must confront all three. pygeoinf does so by working at the level of *abstract spaces and operators*, which makes the structure of the problem explicit and prevents silent, basis-dependent errors.

1.3 Why abstract spaces matter

Why this matters

If you are used to working with matrices and vectors in NumPy, you may wonder why pygeoinf insists on Hilbert spaces, dual pairings, and Riesz maps. This section shows you exactly what goes wrong when you ignore these structures — and how pygeoinf gets it right automatically.

To make the argument concrete, consider a simple 1D problem. You want to recover a function $m : [0, 1] \to \mathbb{R}$ from M integral measurements

$$d_i = \int_0^1 g_i(x)\, m(x)\, \mathrm{d}x, \qquad i = 1, \ldots, M. \tag{1.2}$$

You discretize $[0, 1]$ into N cells with centres x_j^* and widths $\Delta x_j > 0$ (possibly non-uniform). Approximate m as piecewise constant: $m(x) \approx m_j$ on cell j, so $\mathbf{m} = (m_1, \ldots, m_N)^\top \in \mathbb{R}^N$. The discrete forward matrix becomes

$$G_{ij} = g_i(x_j^*)\Delta x_j, \qquad \mathbf{d} = \mathbf{G}\mathbf{m} \in \mathbb{R}^M. \tag{1.3}$$

You want the minimum-norm solution: among all $\mathbf{m}$ with $\mathbf{G}\mathbf{m} = \mathbf{d}$, find the one of smallest norm.

The naive approach: use the Euclidean norm on $\mathbb{R}^N$. Minimizing $\|\mathbf{m}\|_2^2 = \sum_j m_j^2$ gives

$$\mathbf{m}^\dagger_{\text{naive}} = \mathbf{G}^\top(\mathbf{G}\mathbf{G}^\top)^{-1}\mathbf{d}. \tag{1.4}$$

The correct approach: use the $L^2([0,1])$ norm. The natural inner product approximating $\|m\|_{L^2}^2 = \int_0^1 m(x)^2 \, \mathrm{d}x$ is

$$(\mathbf{m}, \mathbf{n})_{L^2} \approx \sum_j m_j n_j \Delta x_j = \mathbf{m}^\top \mathbf{M} \mathbf{n}, \qquad \mathbf{M} = \mathrm{diag}(\Delta x_1, \ldots, \Delta x_N). \tag{1.5}$$

Minimizing this gives the qualitatively different solution

$$\mathbf{m}^\dagger = \mathbf{M}^{-1}\mathbf{G}^\top(\mathbf{G}\mathbf{M}^{-1}\mathbf{G}^\top)^{-1}\mathbf{d}. \tag{1.6}$$

Why they differ. If the grid is uniform ($\Delta x_j = 1/N$ for all j), then $\mathbf{M} = \frac{1}{N}\mathbf{I}$ and the two solutions agree up to a constant. But on a non-uniform grid, the naive solution (1.4) *implicitly weights finely-sampled regions more heavily*: minimizing $\sum_j m_j^2$ without the Δx_j weights is not minimizing a physically meaningful norm of m.

The matrix $\mathbf{M}$ is the *Gram matrix* (or mass matrix) of the basis $\{\phi_j\}$ where $\phi_j = \mathbf{1}_{\text{cell } j}/\Delta x_j^{1/2}$. The formula $G^* = \mathbf{M}^{-1}\mathbf{G}^\top$ is the finite-dimensional instance of the abstract relation

$$G^* = \mathcal{R}_{\mathcal{M}} \circ G' \circ \mathcal{R}_{\mathcal{D}}^{-1}, \tag{1.7}$$

which pygeoinf enforces automatically once you provide the spaces with their correct inner products.

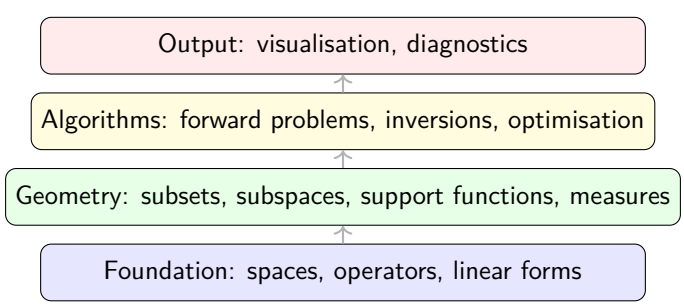
Common Pitfall

The matrix transpose $\mathbf{G}^\top$ is the adjoint only in Euclidean space with the standard dot product. In any other Hilbert space it is not. Using $\mathbf{G}^\top$ instead of $\mathbf{M}_{\mathcal{M}}^{-1}\mathbf{G}^\top\mathbf{M}_{\mathcal{D}}$ produces a solution that is biased toward densely-gridded regions (or, in a Bayesian setting, posterior covariances that are not basis-independent).

This example is the simplest case. As problems become more complex — Sobolev regularity norms, non-trivial data weighting, operators between function spaces — the gap between "naive" and "correct" widens. pygeoinf closes that gap structurally, not by asking the user to remember the right formula each time.

1.4 Package architecture overview

pygeoinf is organized in four ascending layers.



Foundation layer (Part I) The abstract base classes `HilbertSpace`, `LinearOperator`, and `LinearForm` define the mathematical objects. Everything else is built on top of them. No inversion algorithm is aware of the concrete space type; it only calls the abstract interface.

Geometry layer (Part II) Convex subsets (`Ball`, `PolyhedralSet`, ...), subspaces (`AffineSubspace`), support functions (`SupportFunction`), and Gaussian measures (`GaussianMeasure`) all live here. They each accept any `HilbertSpace` and work correctly regardless of its concrete type.

Algorithm layer (Part III) `ForwardProblem`, `LinearBayesianInversion`, `LinearMinimumNormInversion`, `BackusGilbertInversion`, and the convex optimisation algorithms belong here. They compose the foundation and geometry objects into complete solution procedures.

Output layer (Part IV) `SubspaceSlicePlotter` and related tools for visualising results and checking algorithmic correctness.

1.5 How to read this manual

The manual is divided into four parts mirroring the package architecture:

Part I (Foundations) Chapters 2–5. The mathematical vocabulary: spaces, operators, forms. *All users should read at least the motivation sections of these chapters.*

Part II (Geometry) Chapters 6–9. Convex geometry, subspaces, and probabilistic models.

Part III (Problems and Algorithms) Chapters 10–14. How to set up and solve inverse problems.

Part IV (Computational Methods) Chapters 15–16. Numerical backends and visualisation.

Each chapter is self-contained. Forward references are kept to a minimum. The appendices collect a convex analysis primer (Appendix A), functional analysis prerequisites (Appendix B), deferred proofs (Appendix C), and a complete API quick-reference (Appendix D).

Callout boxes.

Yellow — **Why this matters** Motivational content and geophysical perspective.

Blue — **pygeoinf API** The mapping from mathematical concept to Python code.

Green — **Worked Example** Complete mini-examples with both equations and code.

Red — **Common Pitfall** Frequent mistakes and how to avoid them.

1.6 Notation and conventions summary

A complete notation table is given in the Notation and Conventions chapter of the front matter. The most important rules:

- Parentheses $(m_1, m_2)_{\mathcal{M}}$ always denote inner products; angle brackets $\langle \ell, m \rangle$ always denote dual pairings.
- Boldface ($\mathbf{m}$, $\mathbf{G}$) is reserved for finite-dimensional vectors and matrices.
- Calligraphic letters ($\mathcal{M}$, $\mathcal{D}$) denote abstract spaces; Roman capitals (G, T, Π) denote abstract operators.

- G^* always means the *Hilbert* adjoint ($G^* : \mathcal{D} \to \mathcal{M}$); G' means the *Banach* dual ($G' : \mathcal{D}' \to \mathcal{M}'$). They are related by (1.7) but are not the same map.

Chapter 2

Hilbert Spaces

2.1 Why Hilbert spaces?

Why this matters

The inner product on your model space is not a minor technical detail — it determines what "similar models" means, and therefore what your inversion algorithm considers a good solution. Two inversions that differ only in their choice of inner product will in general produce qualitatively different results. pygeoinf makes the inner product an explicit, first-class object so that this choice is always visible and always consistent.

Why not work with plain vectors and matrices? A vector space gives you addition and scalar multiplication, but it has no notion of length or angle. You cannot define a "smallest" element, orthogonality, or projection without more structure. The extra structure needed is an *inner product*.

For geophysical inversion, the inner product on the model space $\mathcal{M}$ is the mathematical encoding of your regularity expectations about the model. A Euclidean inner product treats all degrees of freedom equally, regardless of grid spacing or physical units. An $L^2([0,1])$ inner product weights by area element dx. A Sobolev H^1 inner product additionally penalises gradients. Each choice leads to a different minimum-norm solution; only the "right" one is physically meaningful.

In short: *the inner product is part of the problem specification*, not an implementation detail.

2.2 Definition and axioms

Definition 2.1 (Inner product space)**.** A *real inner product space* is a real vector space H together with a map $(\cdot,\cdot)_H : H \times H \to \mathbb{R}$ satisfying, for all $u, v, w \in H$ and $\alpha \in \mathbb{R}$:

1. **Symmetry:** $(u,v)_H = (v,u)_H$.
2. **Linearity in the first argument:** $(\alpha u + v, w)_H = \alpha(u,w)_H + (v,w)_H$.
3. **Positive definiteness:** $(u,u)_H \geq 0$, with equality only if $u = 0$.

The inner product induces a norm $\|u\|_H := \sqrt{(u,u)_H}$ and hence a metric $d(u,v) = \|u - v\|_H$.

Definition 2.2 (Hilbert space)**.** A real *Hilbert space* is a real inner product space $(H, (\cdot,\cdot)_H)$ that is *complete* with respect to the induced norm: every Cauchy sequence converges to an element of H.

Remark. pygeoinf works exclusively over $\mathbb{R}$. Complex Hilbert spaces are not supported. All spaces are assumed to be *separable*, meaning they admit a countable dense subset; this ensures the existence of countable orthonormal bases.

Orthogonality. Two elements $u, v \in H$ are *orthogonal*, written $u \perp v$, if $(u, v)_H = 0$. A set $S \subset H$ is orthogonal if any two distinct elements of S are orthogonal. If additionally $\|s\|_H = 1$ for all $s \in S$, the set is *orthonormal*.

The parallelogram law. A norm comes from an inner product if and only if it satisfies the *parallelogram law*:

$$\|u+v\|_H^2 + \|u-v\|_H^2 = 2\|u\|_H^2 + 2\|v\|_H^2 \qquad \forall\, u, v \in H. \tag{2.1}$$

The ℓ^p and L^p spaces for $p \neq 2$ are Banach but not Hilbert; they do not satisfy (2.1).

2.3 Inner products, norms, and distances

Proposition 2.3 (Cauchy–Schwarz inequality)**.** *For any $u, v \in H$,*

$$|(u, v)_H| \leq \|u\|_H \, \|v\|_H, \tag{2.2}$$

with equality if and only if u and v are linearly dependent.

The Cauchy–Schwarz inequality guarantees that the cosine of the "angle" between two vectors, $\cos\theta_{u,v} := (u, v)_H/(\|u\|_H\|v\|_H)$, is well-defined and in $[-1, 1]$, even in infinite dimensions.

Polarisation identity. The inner product can be recovered from the norm via

$$(u, v)_H = \tfrac{1}{4}\left(\|u+v\|_H^2 - \|u-v\|_H^2\right). \tag{2.3}$$

This means all metric information about a Hilbert space is encoded in its inner product.

pygeoinf API

`HilbertSpace` exposes:

- `inner_product(x1, x2)`: computes $(x_1, x_2)_H$;
- `norm(x)`: computes $\|x\|_H$;
- `squared_norm(x)`: computes $\|x\|_H^2$ (often cheaper);
- `distance(x1, x2)`: computes $\|x_1 - x_2\|_H$.

Internally, `inner_product` is computed via the dual pairing (see Section 2.4) as $(x_1, x_2)_H = \langle \mathcal{R}^{-1}x_1,\, x_2\rangle$, using `to_dual` and the duality product.

2.4 The dual space and dual pairing

Why this matters

In geophysical terms, a dual vector is exactly what an instrument measures: a number that depends linearly on the model. Understanding the distinction between vectors (models) and dual vectors (measurements) is fundamental to understanding why inversion requires Riesz maps.

Definition 2.4 (Continuous dual space)**.** Let H be a Hilbert space. A *continuous linear functional* on H is a map $\ell : H \to \mathbb{R}$ that is linear ($\ell(\alpha u + v) = \alpha\ell(u) + \ell(v)$) and bounded ($|\ell(u)| \leq C\|u\|_H$ for some $C < \infty$). The collection of all continuous linear functionals on H forms the *(continuous) dual space* H', which is itself a Banach space under the operator norm $\|\ell\|_{H'} := \sup_{\|u\|=1} |\ell(u)|$.

Dual pairing. We denote the evaluation of $\ell \in H'$ at $u \in H$ by the *dual pairing*:

$$\langle \ell, u \rangle := \ell(u). \tag{2.4}$$

We use angle brackets $\langle \cdot, \cdot \rangle$ for dual pairings and parentheses $(\cdot, \cdot)_H$ for inner products throughout this manual; the two are related by the Riesz theorem (Section 2.5).

Two distinct spaces. Elements of H are "primal" vectors (models); elements of H' are "dual" vectors (measurements, functionals, gradients of scalar functions). On a finite-dimensional Euclidean space, $H \cong H'$ via the standard dot product, which is why the distinction is often ignored in numerical practice. On a general Hilbert space — or more acutely, on a Banach space where no inner product exists — the two are different objects and must be handled separately.

Concrete example. Let $H = L^2([0,1])$ and let $g \in L^2([0,1])$. Then the map $\ell : m \mapsto \int_0^1 g(x)m(x)\,\mathrm{d}x$ is a continuous linear functional on H, i.e. $\ell \in H'$. This is precisely the structure of a geophysical measurement: every seismogram, gravity reading, or magnetic datum is a dual vector.

2.5 The Riesz representation theorem

Theorem 2.5 (Riesz representation)**.** *Let H be a real Hilbert space. For every continuous linear functional $\ell \in H'$ there exists a unique $u_\ell \in H$ such that*

$$\ell(v) = (u_\ell,\, v)_H \qquad \forall\, v \in H. \tag{2.5}$$

Moreover, $\|u_\ell\|_H = \|\ell\|_{H'}$. The map $\mathcal{R} : H' \to H$ defined by $\mathcal{R}(\ell) := u_\ell$ is an isometric isomorphism.

Proof sketch. If $\ell = 0$ take $u_\ell = 0$. Otherwise, $\ker \ell$ is a closed proper subspace of H of codimension one. By the projection theorem there exists a unit vector $e \perp \ker \ell$. Set $u_\ell := \ell(e)\, e$. For any $v \in H$, the element $v - \langle \ell, v \rangle e/\ell(e)$ lies in $\ker \ell$, hence is orthogonal to e. A brief calculation shows $(u_\ell, v)_H = \ell(v)$. Uniqueness follows from positive definiteness.

Geometric interpretation. The Riesz representative $u_\ell = \mathcal{R}(\ell)$ is the element of H that "points in the direction" the functional ℓ is measuring. In optimisation this is the *gradient*: if $f(u) = \ell(u)$ then $\nabla f = u_\ell$.

Riesz map and its inverse. We call $\mathcal{R} : H' \to H$ the *Riesz isomorphism* or Riesz map. Its inverse $\mathcal{R}^{-1} : H \to H'$ is defined by $(\mathcal{R}^{-1}u)(v) = (u, v)_H$. Together they establish the identification

$$\langle \ell, v \rangle = (v,\ \mathcal{R}\,\ell)_H \qquad \forall\, \ell \in H',\ v \in H. \tag{2.6}$$

pygeoinf API

The Riesz isomorphism and its inverse are implemented as:

$$\texttt{from_dual} \equiv \mathcal{R} : H' \to H, \qquad \texttt{to_dual} \equiv \mathcal{R}^{-1} : H \to H'.$$

If you need them as `LinearOperator` instances, use the properties `HilbertSpace.riesz` ($\mathcal{R}$) and `HilbertSpace.inverse_riesz` ($\mathcal{R}^{-1}$). The dual pairing evaluates as $\langle \ell, v \rangle = \ell(v)$, implemented by calling the `LinearForm` object directly: `l(v)`.

2.6 Bases and representations

2.6.1 Riesz bases and biorthogonal duals

An *orthonormal basis* (ONB) $\{e_j\}_{j=1}^{\infty}$ is a maximal orthonormal set; every $m \in H$ expands as $m = \sum_j (m, e_j)_H e_j$. But ONBs are rarely computable. A physical or numerical basis $\{\phi_j\}$ is typically not orthonormal, and we must work with general Riesz bases.

Definition 2.6 (Riesz basis)**.** A sequence $\{\phi_j\}_{j=1}^{\infty} \subset H$ is a *Riesz basis* if it is the image of an ONB under a bounded invertible operator. Equivalently, there exist constants $0 < A \le B < \infty$ such that for every finite coefficient sequence $\mathbf{c} \in \mathbb{R}^N$,

$$A\,\|\mathbf{c}\|_{\ell^2}^2 \;\le\; \Big\|\sum_j c_j \phi_j\Big\|_H^2 \;\le\; B\,\|\mathbf{c}\|_{\ell^2}^2. \tag{2.7}$$

Every Riesz basis $\{\phi_j\}$ has a unique *biorthogonal dual* $\{\psi_j\} \subset H$ satisfying

$$(\phi_j, \psi_k)_H = \delta_{jk}. \tag{2.8}$$

Any element $m \in H$ then has the unique expansion

$$m = \sum_{j=1}^{\infty} [\mathbf{m}]_j\, \phi_j, \qquad [\mathbf{m}]_j = (m, \psi_j)_H. \tag{2.9}$$

The coefficient sequence $\mathbf{m} = ([\mathbf{m}]_j)_{j=1}^{\infty}$ lies in ℓ^2, and the norms are equivalent (eq. (2.7)). When $\{\phi_j\}$ is an ONB, $\psi_j = \phi_j$ and $A = B = 1$.

Connection to the dual basis. The dual basis $\{\phi^i\} \subset H'$ is defined by $\langle \phi^i, \phi_j \rangle = \delta^i_j$. The biorthogonal dual vectors are the Riesz representatives of the dual basis:

$$\psi_i = \mathcal{R}(\phi^i). \tag{2.10}$$

2.6.2 The analysis operator

Definition 2.7 (Analysis operator)**.** The *analysis operator* associated with Riesz basis $\{\phi_j\}$ is

$$\Pi : H \to \ell^2, \qquad \Pi(m) := \big[(m, \psi_j)_H\big]_{j=1}^{\infty}. \tag{2.11}$$

The analysis operator extracts the coefficient sequence: $\Pi(m) = \mathbf{m}$. For any Riesz basis it is bounded from H to ℓ^2.

2.6.3 The synthesis operator

Definition 2.8 (Synthesis operator)**.** The *synthesis operator* is

$$\pi : \ell^2 \to H, \qquad \pi(\mathbf{c}) := \sum_{j=1}^{\infty} c_j\, \phi_j. \tag{2.12}$$

The synthesis operator reconstructs m from its coefficients. For a Riesz basis, Π and π are bounded inverses of each other:

$$\pi \circ \Pi = \mathrm{Id}_H, \qquad \Pi \circ \pi = \mathrm{Id}_{\ell^2}. \tag{2.13}$$

pygeoinf API

The coefficient maps correspond to

$$\texttt{to_components} \equiv \Pi : H \to \ell^2, \qquad \texttt{from_components} \equiv \pi : \ell^2 \to H.$$

In practice a concrete space has finite `dim` $= N$, giving truncated maps $\Pi_N : H \to \mathbb{R}^N$ and $\pi_N : \mathbb{R}^N \to H_N$. The operators `coordinate_projection` and `coordinate_inclusion` expose Π_N and π_N as `LinearOperator` instances (including their adjoints).

2.6.4 The dual basis

On the dual space H' the *dual basis* $\{\phi^i\}_{i=1}^{\infty}$ is defined by $\langle \phi^i, \phi_j \rangle = \delta^i_j$. It forms a Riesz basis for H'. The analysis operator on H' is $\Pi' : H' \to \ell^2$, $[\Pi'\ell]_j = \langle \ell, \phi_j \rangle$, and the synthesis operator on H' is $\pi' : \ell^2 \to H'$, $\pi'(\mathbf{c}) = \sum_j c_j \phi^j$.

The key identity connecting dual and primal bases is:

$$(\pi')^* = \Pi, \qquad (\Pi')^* = \pi, \tag{2.14}$$

where $(\pi')^* : H \to \ell^2$ and $(\Pi')^* : \ell^2 \to H$ denote the Hilbert adjoints of π' and Π' with respect to the inner products on H and ℓ^2.

2.7 The Gram matrix

Definition 2.9 (Gram operator / mass matrix)**.** Let $\{\phi_j\}$ be a Riesz basis for H. The *Gram operator* (or *mass operator*) is

$$\mathbf{M}_\phi := \pi^* \pi : \ell^2 \to \ell^2, \qquad [\mathbf{M}_\phi]_{jk} = (\phi_k, \phi_j)_H. \tag{2.15}$$

In finite dimensions (N-truncation) it becomes the $N \times N$ matrix

$$[\mathbf{M}_{\phi,N}]_{jk} = (\phi_k, \phi_j)_H, \qquad j, k = 1, \ldots, N. \tag{2.16}$$

Proposition 2.10. *For any Riesz basis, $\mathbf{M}_\phi$ is bounded, self-adjoint, and strictly positive on ℓ^2. In particular it is invertible, and its finite-dimensional truncation $\mathbf{M}_{\phi,N}$ is symmetric positive definite (SPD) for every $N \geq 1$.*

Gram matrix and the biorthogonal dual. The biorthogonal dual basis is related to the original basis by

$$\psi_j = \sum_i [\mathbf{M}_\phi^{-1}]_{ji}\, \phi_i. \tag{2.17}$$

This gives a purely computational recipe: to find ψ_j, solve the linear system $\mathbf{M}_\phi \mathbf{c} = \mathbf{e}_j$ (where $\mathbf{e}_j$ is the j-th standard basis vector) and form $\sum_i c_i \phi_i$.

Analysis via ϕ_j alone. Substituting (2.17) into the definition of Π gives

$$\Pi = \mathbf{M}_\phi^{-1}\, \pi^* : H \to \ell^2. \tag{2.18}$$

This means the analysis operator can be computed purely from $\{\phi_j\}$ — the biorthogonal dual need not be constructed explicitly. One forms the matrix $\mathbf{M}_{\phi,N}$, inverts it, and multiplies the vector of inner products $[\langle \phi_j, m\rangle]_j$.

pygeoinf API

In pygeoinf, `HilbertSpace.coordinate_projection` implements $\Pi_N : H \to \mathbb{R}^N$ via (2.18): it evaluates $[\pi_N^*(m)]_j = (\phi_j, m)_H$ and then multiplies by $\mathbf{M}_{\phi,N}^{-1}$. The Gram matrix itself is available via `HilbertSpace.gram_matrix(N)`.

Riesz isomorphism on the coefficient space. On the finite coefficient space $\mathbb{R}^N$ (equipped with the norm inherited from H via π_N), the Riesz maps are given by the Gram matrix:

$$\mathcal{R}_{\mathbb{R}^N}(\mathbf{a}) = \mathbf{M}_{\phi,N}^{-1}\mathbf{a}, \qquad \mathcal{R}_{\mathbb{R}^N}^{-1}(\mathbf{u}) = \mathbf{M}_{\phi,N}\, \mathbf{u}. \tag{2.19}$$

This explains the ubiquity of the Gram matrix in inversion formulas: it converts coefficient vectors between the primal and dual representations.

2.8 Finite-dimensional approximation

All computation in pygeoinf is finite-dimensional. For a Riesz basis $\{\phi_j\}_{j=1}^\infty$ we fix a truncation parameter N and define:

$$H_N := \operatorname{span}\{\phi_1, \ldots, \phi_N\} \subset H. \tag{2.20}$$

The *truncated analysis and synthesis operators* are

$$\Pi_N : H \to \mathbb{R}^N, \quad \Pi_N(m) = \big[(m, \psi_j)_H\big]_{j=1}^N, \tag{2.21}$$

$$\pi_N : \mathbb{R}^N \to H_N, \quad \pi_N(\mathbf{c}) = \sum_{j=1}^N c_j\, \phi_j. \tag{2.22}$$

The *orthogonal projection* onto H_N is

$$P_N := \pi_N \Pi_N : H \to H_N, \tag{2.23}$$

which satisfies $(m - P_N m, \psi_j)_H = 0$ for all $j \leq N$. As $N \to \infty$, $\|m - P_N m\|_H \to 0$ for every $m \in H$.

Why this matters

The projection P_N is the fundamental link between infinite-dimensional theory and finite computation. Every algorithm in pygeoinf ultimately operates on $\mathbb{R}^N$ via the maps Π_N and π_N; the theory guarantees that as N grows the computed results converge to the infinite-dimensional truth.

2.9 Concrete spaces in pygeoinf

This section introduces the built-in Hilbert space implementations and explains how to create your own.

2.9.1 `EuclideanSpace`

$\mathbb{R}^n$ with the standard inner product $(u, v) = \sum_i u_i v_i = \mathbf{u}^\top \mathbf{v}$. The Riesz map is the identity: $\mathcal{R} = \mathrm{Id}$. The basis $\{\phi_j\} = \{e_j\}$ is the standard ONB, so $\mathbf{M}_\phi = \mathbf{I}$ and $\Pi = \pi^* = \mathrm{Id}$.

When you use `EuclideanSpace(n)`, adjacent operator transposes *are* adjoints, and you recover the classical formulas of linear algebra. This is the appropriate choice when variables represent coefficients in a pre-orthonormalised basis (e.g. after a change of variables $\tilde{m} = \mathbf{L}m$ where $\mathbf{L}$ is a Cholesky factor of $\mathbf{M}_\phi$).

2.9.2 `MassWeightedHilbertSpace`

Given a Hilbert space H and a bounded symmetric positive operator $\mathbf{W} : H \to H'$ (the "weight operator"), the *mass-weighted space* $H_\mathbf{W}$ is the same underlying vector space but with inner product

$$(u, v)_{H_\mathbf{W}} = \langle \mathbf{W}u,\, v \rangle. \tag{2.24}$$

The Riesz maps become $\mathcal{R}_\mathbf{W} = \mathcal{R} \circ \mathbf{W}$ and $\mathcal{R}_\mathbf{W}^{-1} = \mathbf{W}^{-1} \circ \mathcal{R}^{-1}$.

Motivating use case: discretized L^2. Using the non-uniform grid inner product from Section 1.3, we set $H = \mathbb{R}^N$, $H_\mathbf{W} = \mathbb{R}^N$ with weight $\mathbf{W} = \mathbf{M} = \mathrm{diag}(\Delta x_j)$. Then $(u, v)_{H_\mathbf{W}} = \sum_j u_j v_j \Delta x_j \approx \int_0^1 u(x)v(x)\,\mathrm{d}x$.

2.9.3 `HilbertSpaceDirectSum`

Given Hilbert spaces H_1 and H_2, their *Hilbert direct sum* $H_1 \oplus H_2$ consists of pairs (u_1, u_2) with

$$\big((u_1, u_2),\, (v_1, v_2)\big)_{H_1 \oplus H_2} = (u_1, v_1)_{H_1} + (u_2, v_2)_{H_2}. \tag{2.25}$$

The Riesz maps and analysis/synthesis operators decompose blockwise. This is the natural structure for models that are concatenations of separately-structured fields (e.g. P-wave and S-wave velocity perturbations).

2.9.4 Implementing your own `HilbertSpace`

To encode a non-standard model space, subclass `HilbertSpace`. The abstract interface requires exactly eight methods:

pygeoinf API

Subclass `HilbertSpace` and implement:

- `dim` — integer dimension N of the space.
- `zero()` — return the zero element.
- `basis_vector(j)` — return ϕ_j for $j = 0, \ldots, N-1$.
- `inner_product(x1, x2)` — compute $(x_1, x_2)_H$.
- `to_dual(x)` — compute $\mathcal{R}^{-1}(x) \in H'$.
- `from_dual(xp)` — compute $\mathcal{R}(\ell) \in H$ for $\ell \in H'$.
- `to_components(x)` — compute $\Pi(x) \in \mathbb{R}^N$.
- `from_components(c)` — compute $\pi(\mathbf{c}) \in H$.

All other methods (`norm`, `distance`, `project`, `riesz`, `gram_matrix`, `coordinate_projection`, `coordinate_inclusion`, ...) are derived automatically from the above.

Worked Example

Trapezoidal L^2 approximation. Suppose your model lives in $L^2([0,1])$ and you discretize using a non-uniform grid $0 = x_0 < x_1 < \cdots < x_N = 1$ with midpoints $x_j^* = (x_{j-1} + x_j)/2$ and widths $\Delta x_j = x_j - x_{j-1}$. Implement:

```
class NonUniformL2(HilbertSpace):
    def __init__(self, x):
        self._x = np.asarray(x)          # grid points
        self._dx = np.diff(x)            # cell widths Delta x_j
    @property
    def dim(self):
        return len(self._dx)
    def inner_product(self, u, v):
        return np.dot(u * self._dx, v)   # sum u_j v_j Delta x_j
    def to_dual(self, u):
        return u * self._dx              # (R^-1 u)_j = u_j Delta x_j
    def from_dual(self, up):
        return up / self._dx             # (R ell)_j = ell_j / Delta
            x_j
    def to_components(self, u):
        return u.copy()                  # basis = piecewise constant
    def from_components(self, c):
        return c.copy()
    def zero(self):
        return np.zeros(self.dim)
    def basis_vector(self, j):
        e = np.zeros(self.dim); e[j] = 1.0; return e
```

With this space, `to_dual` implements the mass-weighting that equation (1.6) requires: the adjoint formula becomes $\mathbf{G}^* = \mathbf{M}^{-1}\mathbf{G}^\top\mathbf{M}_{\mathcal{D}}$ automatically.

2.10 Summary and commutative diagram

All the maps introduced in this chapter fit together in a single commutative diagram. Let H be a Hilbert space with Riesz basis $\{\phi_j\}_{j=1}^N$ (finite-dimensional for conciseness). The spaces, dual spaces, coefficient space $\mathbb{R}^N$, its dual, and all the maps between them commute as follows:

$$\begin{tikzcd}
H \arrow[r, bend right, "\Pi_N"'] \arrow[d, leftrightarrow, "\mathcal{R}"'] & \mathbb{R}^N \arrow[l, bend right, "\pi_N"'] \arrow[d, leftrightarrow, "\mathbf{M}_{\phi,N}"] \\
H' \arrow[r, bend right, "(\Pi_N)^*"'] & (\mathbb{R}^N)' \arrow[l, bend right, "\pi_N^*"']
\end{tikzcd}$$

Reading the diagram:

- The blue horizontal arrows are the analysis (Π_N) and synthesis (π_N) operators and their duals.
- The vertical arrows on the left are the Riesz maps for H.
- The vertical arrows on the right are the Gram matrix and its inverse, corresponding to the Riesz maps for $\mathbb{R}^N$ equipped with the basis inner product.
- Every path between two nodes gives the same result: the diagram commutes.

The outer square expresses $\mathcal{R}^{-1} = \pi_N^* \circ \mathbf{M}_{\phi,N} \circ \Pi_N$, which is the coordinate representation of the Riesz map. In Chapter 4 this diagram will be extended to include a forward operator $G : \mathcal{M} \to \mathcal{D}$ and its adjoint G^*, each appearing as arrows connecting two copies of the above diagram (one for $\mathcal{M}$, one for $\mathcal{D}$).

Chapter 3

Linear Forms

3.1 Motivation: measurements as linear functionals

Why this matters

Every single datum in your data vector is the result of applying a linear functional to the model. Understanding linear forms is understanding what your data actually *are* mathematically.

3.2 Definition

3.3 Representation in coordinates

3.4 The Riesz representative (gradient)

3.5 Linear forms as rank-1 operators

pygeoinf API

`LinearForm` and `DualVector` represent elements of $\mathcal{M}'$. Key methods: `__call__(x)`, `components`, `as_linear_operator()`.

Chapter 4

Linear Operators

4.1 Motivation: the forward operator

Why this matters

The forward operator G is the mathematical encoding of your physics. Getting its adjoint right is critical for every inversion algorithm — and the adjoint depends on the inner products of both spaces, not just the matrix transpose.

4.2 Definition and properties

4.3 The Banach dual (transpose) G'

4.4 The Hilbert adjoint G^*

Common Pitfall

If you are used to thinking of the matrix transpose as the adjoint, beware: that is only true in Euclidean space with the standard inner product. For general Hilbert spaces, $G^* \neq G^\top$. The correct relationship is $G^* = \mathcal{R}_{\mathcal{M}} \circ G' \circ \mathcal{R}_{\mathcal{D}}^{-1}$.

4.5 Matrix representations

4.5.1 The standard (component) matrix

4.5.2 The Galerkin matrix

4.5.3 The discrete adjoint formula

4.6 Operators as stacks of linear forms

pygeoinf API

Given linear forms $\ell_i \in \mathcal{M}'$, construct the forward operator via `LinearOperator.from_linear_forms([`$\ell_1, \ldots, \ell_M$

4.7 Kernel and range

4.8 Constructing operators in pygeoinf

pygeoinf API

`LinearOperator(domain, codomain, mapping)` is the primary constructor. Optionally provide `dual_mapping` (G') or `adjoint_mapping` (G^*); the library derives the other via the Riesz maps.

4.9 The full commutative diagram

Chapter 5

Nonlinear Operators and Forms

5.1 NonLinearOperator

5.2 NonLinearForm

5.3 Gradients and Hessians

Part II

Geometry and Structure

Chapter 6

Subsets and Convex Sets

6.1 Why convex sets?

Why this matters

Every piece of prior information you have about your model — bounds on parameters, energy constraints, smoothness requirements — defines a set of admissible models. Convexity is the mathematical property that makes these constraints computationally tractable.

6.2 The Subset interface

6.3 Balls and ellipsoids

6.4 Half-spaces and polyhedral sets

6.5 Degenerate sets: EmptySet and FullSpace

6.6 Set operations

Chapter 7

Subspaces

7.1 Motivation: equality constraints

7.2 Orthogonal projectors

pygeoinf API

`OrthogonalProjector.from_basis(space, [v1, v2, ...])` constructs the projection $P = \sum_i \hat{v}_i \hat{v}_i^\top$ from spanning vectors.

7.3 Linear subspaces

7.4 Affine subspaces

7.4.1 Construction from tangent bases

7.4.2 Construction from linear equations

7.5 Tangent spaces and dimension

Chapter 8

Convex Analysis and Support Functions

Why this matters

Convex analysis provides a complete, probability-free framework for answering the question: given my data and prior constraints, what can I confidently say about the model? The support function is the central object that makes this possible.

8.2 Support functions: intuition

8.3 Formal definition and properties

8.4 Subgradients and support points

8.5 Concrete support functions

8.5.1 BallSupportFunction

8.5.2 EllipsoidSupportFunction

8.5.3 PolyhedralSupportFunction

8.5.4 SobolevBallSupportFunction

8.6 Combinators: building complex sets from simple ones

8.6.1 Infimal convolution (Minkowski sum)

8.6.2 Support function sum (intersection)

8.6.3 Scaled sums

8.7 The dual master equation

Chapter 9

Gaussian Measures

9.1 Why Gaussian measures?

Why this matters

If your prior knowledge about the model is naturally described by a mean and covariance, and your forward operator is linear, and your measurement noise is Gaussian, then the full posterior distribution is Gaussian and can be computed in closed form. This is what `GaussianMeasure` provides.

9.2 Gaussian measures on Hilbert spaces

9.3 Sampling

9.4 Push-forward and marginals

9.5 Conditioning (Bayesian update)

Part III

Problems and Algorithms

Chapter 10

Forward Problems

10.1 The forward modelling framework

10.2 ForwardProblem and LinearForwardProblem

pygeoinf API

`LinearForwardProblem(model_space, data_space, forward_operator, data_error_measure)` bundles the complete forward problem specification.

Chapter 11

Deterministic Inversion

11.1 Least-squares inversion

11.2 Minimum-norm inversion

Why this matters

The minimum-norm solution depends on the inner product: change the norm, change the answer. This is why the careful treatment of Hilbert spaces in Chapter 2 pays off here.

11.3 Constrained inversions

11.4 Connection to the dual master equation

Chapter 12

Bayesian Inversion

12.1 The Bayesian framework for inverse problems

12.2 Linear Bayesian inversion

pygeoinf API

`LinearBayesianInversion(forward_problem, prior)` computes the posterior Gaussian measure. Access via `.invert(data)`.

12.3 Constrained Bayesian inversion

12.4 Posterior uncertainty

Chapter 13

Backus–Gilbert and DLI Inference

13.1 The inference problem

13.2 Backus–Gilbert estimators

13.3 The admissible property set $\mathcal{U}$

13.4 Computing $\mathcal{U}$ via the dual master equation

13.5 Polytope outer approximation

Chapter 14

Convex Optimisation

14.1 Overview of optimisation in pygeoinf

14.2 Linear solvers

14.3 Convex optimisation algorithms

14.4 Preconditioners

Part IV

Computational Methods and Visualisation

Chapter 15

Computational Utilities

15.1 Randomised matrix decompositions

15.2 Direct sum operators

15.3 Parallel computation

15.4 Auxiliary functions

Chapter 16

Visualisation

16.1 The SubspaceSlicePlotter

16.2 1D slices

16.3 2D slices

16.4 3D slices

16.5 Worked example: visualising the admissible property set

Appendices

Appendix A

Convex Analysis Primer

A.1 Convex sets and functions

A.2 Separation theorems

A.3 Conjugate (Fenchel–Legendre) duality

A.4 Subdifferentials

A.5 Infimal convolution

Appendix B

Functional Analysis Prerequisites

B.1 Banach spaces

B.2 Weak and weak-* topologies

B.3 Compact operators

B.4 Spectral theory for self-adjoint operators

Appendix C

Deferred Proofs

Appendix D

Complete API Quick Reference

D.1 hilbert_space module

D.2 linear_forms module

D.3 linear_operators module

D.4 subsets module

D.5 subspaces module

D.6 convex_analysis module

D.7 gaussian_measure module

D.8 forward_problem module

D.9 inversion module

D.10 linear_bayesian module

D.11 linear_optimisation module

D.12 backus_gilbert module

D.13 visualisation module

Bibliography

[1] David Al-Attar and Ophelia Crawford. Linear inference problems with deterministic constraints. *Geophysical Journal International*, 2021. doi: 10.1093/gji/ggab414.

[2] George Backus. Inference from inadequate and inaccurate data, I. *Proceedings of the National Academy of Sciences*, 65(1):1–7, 1970. doi: 10.1073/pnas.65.1.1.

[3] George Backus. Inference from inadequate and inaccurate data, III. *Proceedings of the National Academy of Sciences*, 67(1):282–289, 1970. doi: 10.1073/pnas.67.1.282.

[4] George Backus. Inference from inadequate and inaccurate data, II. *Proceedings of the National Academy of Sciences*, 65(2):281–287, 1970. doi: 10.1073/pnas.65.2.281.

[5] Andrew M. Stuart. Inverse problems: A Bayesian perspective. *Acta Numerica*, 19:451–559, 2010. doi: 10.1017/S0962492910000061.